

Automated Vulnerability Discovery & Remediation Pipeline

Author: Iulian Igas

Date:

GitHub repository: https://github.com/iulianigas/DSO-lab

Doc
[https://github.com/iulianigas/DSO-lab]
ker/iulianigas/DSO-lab

Executive Summary

This project implements a shift-left DevSecOps workflow for a containerized WordPress deployment. We deployed an intentionally outdated baseline, automated discovery with WPScan in GitHub Actions, remediated through core upgrades and container hardening, published a fixed image to GitHub Container Registry (GHCR) and Docker Hub, and verified improvements via automated rescanning.

Outcome: WordPress core upgraded from **5.8.3** to **6.7.2**, known CVE exposure reduced from **14** reported issues to **0**, user enumeration blocked, debug mode disabled, and Apache/PHP attack surface reduced.

Project flow (step by step)

1. **Deploy vulnerable WordPress** to `docker/docker-compose.yml` (WordPress 5.8.3, MySQL 5.7).

2. **Remediate**

Required deliverables mapping

A. GitHub Repository

Requirement	Location
-------------	----------

|-----|

B. Docker Hub Repository

Requirement	Location
-------------	----------

|-----|

C. PDF report sections

This document covers all required sections (1-6) below. Export: `python3 scripts/generate-pdf.py`.

1. Environment Setup

1.1 Steps taken

1. Created [docker/docker-compose.yml](https://github.com/iulianigas/DSO-lab/blob/main/docker/docker-compose.yml) with WordPress 5.8.3 and MySQL 5.7 (lab baseline).

2. Sta

1.2 Challenges encountered and solutions

Challenge	Solution
-----------	----------

|-----

2. Findings Overview

2.1 Key vulnerabilities

Finding	Risk	CVSS (indicative)
---------	------	-------------------

|-----

2.2 Risk assessment

- Likelihood: High ? WordPress 5.8.x is actively targeted; public exploits exist for multiple listed CVEs.

- Impact

2.3 How attackers could exploit them

1. Authenticated SQL injection (WP < 6.0.3): Contributor+ crafts malicious post meta ? extract wp_users.user_pass hashes ? offline crack ? admin takeover.

2. wp-lo

Evidence: scans/before-remediation-20250520T120000Z.txt (and GitHub Actions artifacts from scan.yml runs).

3. Remediation Steps

3.1 Patching and updating

Component	Before	After
-----------	--------	-------

3.2 Hardening measures

- **Non-root runtime:** Container runs as `www-data`; Apache listens on **8080**.

3.3 Before/after scan evidence

Metric	Before	After
--------	--------	-------

Files: `scans/before-remediation-*` vs `scans/after-remediation-*`

4. Fixed Image Build

Resource	URL
----------	-----

Pull commands:

```
docker pull iulianigas/wordpress-hardened:latest
```

Build locally:

```
docker build -f dockerfiles/Dockerfile.hardened -t iulianigas/wordpress-hardened:latest .
```

5. Tooling Justification

Tool	Role	Why
------	------	-----

6. DevSecOps Strategy (shift-left security)

Security is shifted **left** (earlier in the lifecycle) because:

1. **Scans run before production** ? every push to `main` triggers `scan.yml` against an ephemeral vulnerable stack.
2. **F**

Workflow summary (assignment steps 1-8)

Step	Action	

Appendix ? Commit history (remediation narrative)

View	full	history:
https://github.com/iulianigas/DSO-lab/commits/main		
1. <code>feat: initial vulnerable WordPress docker-compose deployment</code>		
2. <code>ci:</code>		

End of report